

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
            numbers)
    )
)
```

```
print(result)
```

- A)[1, 3, 5]
- B)[4, 8]
- C)[2, 4, 6, 8, 10]
- D)[2, 4]

Q2. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

- A)6.0
- B)36
- C)NameError
- D)6

Q3. What happens when this code executes?

```
def fun():
    print(variable)
    variable = 50
```

```
variable = 10
```

```
fun()
```

- A)UnboundLocalError
- B)10
- C)NameError
- D)50

Q4. What happens here?

```
try:
```

```
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

- A)ValueError
- B)e
- C)Wrong Value
- D)Nothing

Q5. What is the output?

```
def fun():
    try:
        return 10
    finally:
        return 20
```

```
print(fun())
```

- A)20
- B)10
- C)RuntimeError
- D)None

Q6. What is the output?

```
try:
    print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")
```

- A)A
- B)A B
- C)B
- D)SyntaxError

Q7. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

- A)5
- B)5.0
- C)25
- D)sqrt(25)



Q8. What is the output?

```
def outer():  
    x = 10  
  
    def inner():  
        nonlocal x  
        x = x + 5  
        return x  
  
    inner()  
    return x
```

print(outer())

- A)10
- B)UnboundLocalError
- C)15
- D)5

Q9. What will normally happen?

```
import math  
import math
```

- A)ImportError
- B)The module is normally loaded once and reused
- C)SyntaxError
- D)The module is normally loaded and executed twice

Q10. Which object keeps track of modules that have already been imported during the current Python process?

- A)sys.path
- B)sys.imports
- C)sys.modules
- D).__module__

Q11. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```

print(fun(3))

- A)6
- B)None
- C)3
- D)5

Q12. What will be printed?

```
class InvalidAgeError(Exception):  
    pass  
  
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is too low")  
    return "Allowed"
```

try:

```
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

- A)Exception
- Age is too low
- B)Nothing, because custom exceptions cannot be caught
- C)InvalidAgeError
- Age is too low
- D)InvalidAgeError
- 15

Q13. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

try:

```
    first()  
except RuntimeError:  
    print("Handled")
```

- A)NameError
- B)Handled
- C)RuntimeError cannot propagate through functions
- D)TypeError



Q14. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

```
print(result)
```

- A)9
- B)24
- C)10
- D)0

Q15. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)  
print(b)
```

- A)[10, 10]
- [10]
- B)[10]
- [10, 10]
- C)[10]
- [10]
- D)Error

Q16. if test.py has

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If test.py is executed directly, what will be printed?

- A)Nothing
- B)A followed by B
- C)A
- D)B

Q17. What is the output?

```
def fun(x):
```

```
    x += 10  
    return x
```

```
x = 5  
y = fun(x)
```

```
print(x, y)
```

- A)5 5
- B)5 15
- C)15 5
- D)15 15

Q18. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

- A)Both import only sqrt
- B)First provides the module namespace; second imports sqrt directly
- C)First imports only sqrt
- D)There is no difference

Q19. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

- A)20
- B)SyntaxError
- C)30
- D)TypeError

Q20. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

- A)5
- B)15
- C>Error
- D)10

Q21. What is the output?

```
def fun(x):  
    return x * 2
```



```
a = fun
print(a(5))
```

- A)func(5)
- B)5
- C)Error
- D)10

Q22. What does the following function return?

```
def fun():
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A)Hello None
- B)Error
- C)Hello
- D)None

Q23. What will be printed?

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("A")
except Exception:
    print("B")
```

- A)B
- B)A B
- C)A
- D)Nothing

Q24. What will happen?

```
def outer():
    x = 10

    def inner():
        nonlocal y
        y = 20
```

```
    inner()
```

```
outer()
```

- A)20
- B)10
- C)NameError
- D)SyntaxError

Q25. What is the output?

```
def fun(a=10, b=20):
    return a - b
```

```
print(fun(30))
```

- A)30
- B)-10
- C)10
- D)20

Q26. What will be printed?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, b=20))
```

- A)20
- B)30
- C)TypeError
- D)SyntaxError

Q27. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

- A)A
- B)Nothing
- C)A and B
- D)B

Q28. What will be printed?

```
def fun():
    try:
        return 10
    finally:
        print("Finally")
```

```
print(fun())
```

- A)10 only
- B)Finally only
- C)Finally followed by 10



D)10 followed by Finally

Q29. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

print(fun(-5))

A)negative

cleanup

B)cleanup

100

C)100

cleanup

D)cleanup

-5

Q30. What will be printed?

x = 10

f = lambda: x

x = 20

print(f())

A)None

B)20

C)NameError

D)10

Q31. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

fun()

A)Nothing

B)B

C)A

D)A B

Q32. What will be the output?

x = 10

```
def fun():  
    print(x)  
    x = 20
```

fun()

A)UnboundLocalError

B)20

C)NameError

D)10

Q33. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

return counter

a = make_counter()

b = make_counter()

print(a())

print(a())

print(b())

print(a())

A)0

1

0

2

B)1

2

1

3

C)1

2

3

4

D)1

1

1

1



Q34. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

A)NameError

B)10

C)20

D)30

Q35. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        return x + 5
```

```
    return inner
```

```
f = outer()
```

```
print(f())
```

A)15

B)None

C)10

D)NameError

Q36. What happens when this function is called?

```
def fun(a=10, b):
```

```
    return a + b
```

A)It executes correctly

B)SyntaxError

C)NameError

D)TypeError

Q37. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

A)random cannot be imported

B)Python will always reject it

C)Python automatically renames the local file

D)The local file can shadow the intended standard-library module

Q38. What happens here?

```
def fun(a):
```

```
    return a
```

```
print(fun())
```

A)0

B)None

C)NameError

D)TypeError

Q39. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A

B)B

C)B C

D)A C

Q40. What will be the output?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        global x
```

```
        x = 30
```

```
    inner()
```

```
    print(x)
```

```
outer()
```

```
print(x)
```



A)20
10
B)20
30
C)30
20
D)30
30

Q41. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

codes's second handler never reached?

A)ZeroDivisionError occurs before Exception
B)ZeroDivisionError is not an exception
C)except blocks execute randomly
D)Exception is a superclass of ZeroDivisionError

Q42. Suppose sample.py contains:

```
x = 10
```

```
def display():
```

```
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
```

```
sample.display()
```

A)x
display
B)Only Hello
C)10
Hello
D>Error

Q43. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It prevents syntax errors
B)It creates the main function

C)It prevents all imports

D)It identifies code intended to run when the file is executed directly

Q44. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():
```

```
    raise MyError("Something went wrong")
```

```
try:
```

```
    test()
```

```
except MyError as e:
```

```
    print(type(e).__name__)
```

A)Something went wrong

B)Exception

C)TypeError

D)MyError

Q45. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
```

```
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)ImportError

B)Module loaded

Module loaded

10

C)10

10

D)Module loaded

10

Q46. What is the output?

```
def fun():
```

```
    try:
```



```

    print("A")
    raise ValueError
except ValueError:
    print("B")
    return 10
else:
    print("C")
    return 20
finally:
    print("D")

```

print(fun())

- A)A
- B
- D
- 10
- B)A
- B
- 10
- D
- C)A
- C
- D
- 20
- D)A
- B
- C
- D
- 10

Q47. What will happen?

```

def fun(a, b=10):
    return a + b

```

print(fun(b=5, a=20))

- A)15
- B)TypeError
- C)25
- D)SyntaxError

Q48. What is the output?

```

try:
    x = int("abc")
except ValueError:
    print("A")
else:
    print("B")

```

```

finally:
    print("C")

```

- A)B C
- B)B
- C)A
- D)A C

Q49. What is the output?

```

try:
    raise ValueError
except ValueError:
    print("A")
    raise

```

- A)B
- B)A, and execution ends normally
- C)No Output
- D)A, followed by propagation of the same exception

Q50. What will happen?

```

def fun(n):
    if n == 0:
        return 1
    return n * fun(n)

```

print(fun(3))

- A)6
- B)3
- C)TypeError
- D)RecursionError

